

ERD Generator User Guide

Entity Relationship Diagram Generator v1.0

Table of Contents

1. [Overview](#)
 2. [How It Fits Into the Workflow](#)
 3. [Installation](#)
 4. [Using the GUI](#)
 5. [Output Formats](#)
 6. [Color Schemes \(Draw.io\)](#)
 7. [Working with Output Files](#)
 8. [Command-Line / API Usage](#)
 9. [Troubleshooting](#)
-

Overview

ERD Generator reads the HTML documentation produced by **DbDocGen** and turns it into a visual Entity-Relationship Diagram. It parses table definitions, column details, primary keys, and foreign key relationships from the HTML file, then renders a clean diagram in the format of your choice.

How It Fits Into the Workflow

Database → DbDocGen → HTML documentation → ERD Generator → Diagram

1. Use **DbDocGen** to connect to your database and export HTML documentation.
 2. Point **ERD Generator** at that HTML file to produce a diagram.
-

Installation

Prerequisites

- Python 3.7 or higher
- The Graphviz **system binary** (required for PNG, SVG, and PDF output)

Step 1 — Install Python Packages

```
pip install -r requirements.txt
```

Or install directly:

```
pip install beautifulsoup4 graphviz lxml
```

Step 2 — Install the Graphviz System Binary

The Python `graphviz` package is only a wrapper — the Graphviz rendering engine must also be installed on your system.

Windows

Option A — Chocolatey (easiest)

```
choco install graphviz
```

Option B — Manual installer

1. Download the installer from graphviz.org/download/
2. Run the installer and check “Add Graphviz to the system PATH for all users”
3. Open a **new** terminal and verify: `dot -V`

Linux (Ubuntu / Debian)

```
sudo apt-get install graphviz
```

macOS

```
brew install graphviz
```

Note: [Draw.io](https://draw.io), Lucidchart, and Microsoft Visio formats do **not** require the Graphviz binary — they are generated as XML or SVG directly.

Step 3 — Verify Installation

```
python test_installation.py
```

This checks all dependencies including the Graphviz binary and prints a pass/fail result for each.

Using the GUI

Launch the application from the `ERDGenerator` directory:

```
python erdgen.py
```

The window has four controls:

1. Input HTML File

Click **Browse...** and select the HTML file generated by DbDocGen (e.g., `mydb_documentation.html`).

2. Output Format

Choose from the dropdown (see [Output Formats](#) for guidance on which to pick).

3. Color Scheme

Visible only when **DRAWIO** is selected. Choose a header color for the table boxes in the diagram (see [Color Schemes](#)).

4. Output Location

Click **Browse...** and specify where to save the diagram file. The file extension is set automatically based on the selected format.

Generating

Click **Generate ERD**. The status log at the bottom of the window shows progress. A confirmation message appears when the file has been written successfully.

Output Formats

Format	Extension	Best Used For	Requires Graphviz
PNG	.png	Documents, presentations, embedding in Word/PowerPoint	Yes
SVG	.svg	Web pages, wikis, large schemas (scales without blurring)	Yes
PDF	.pdf	Printing, sharing, archiving	Yes
DRAWIO	.drawio	Editing and customising the diagram in Draw.io / diagrams.net	No
Lucidchart	.svg	Importing into Lucidchart for team collaboration	No
Microsoft Visio	.svg	Importing into Microsoft Visio	No

Tips:

- Use **PNG** for most documentation and presentations.
- Use **SVG** when the database has many tables — it scales to any zoom level without quality loss.
- Use **DRAWIO** when you want to rearrange tables, add annotations, or change styling after generation.
- Use **PDF** for the highest-quality print output.

Color Schemes ([Draw.io](#))

When the **DRAWIO** format is selected, a **Color Scheme** dropdown appears. This controls the background colour of table header rows in the diagram.

Scheme	Header Colour
Blue <i>(default)</i>	#4472C4 — corporate blue
Green	#70AD47 — medium green
Orange	#ED7D31 — orange
Purple	#7030A0 — purple
Red	#C00000 — deep red
Teal	#0097A7 — teal
Gray	#5A5A5A — dark gray
Pastel Blue	#B4C7E7 — light blue (black text)
Pastel Green	#C5E0B4 — light green (black text)
Dark	#1F1F1F — near-black

Working with Output Files

PNG / SVG / PDF

Open the file directly in any image viewer, browser, or PDF reader. Embed it in documents or wikis using a standard image reference.

Embedding SVG in HTML

```

```

Draw.io (.drawio)

1. Go to app.diagrams.net (or open the desktop app).
2. **File** → **Open** and select the `.drawio` file.
3. Rearrange tables, edit labels, add swim lanes, or change colours.
4. Export as PNG, SVG, or PDF from **File** → **Export As**.

You can also add the `.drawio` file to a GitHub or GitLab repository — both platforms render it automatically.

Lucidchart

1. In Lucidchart, go to **File** → **Import**.
2. Select the SVG file produced by ERD Generator.
3. Adjust the layout and share with your team.

Microsoft Visio

1. In Visio, go to **File** → **Open** and select the SVG file.
2. Visio imports it as a drawing you can edit further.

Command-Line / API Usage

You can drive ERD Generator from Python without the GUI:

```
from erd_html_parser import HTMLSchemaParser
from erd_diagram_generator import ERDGenerator

# 1. Parse the HTML documentation produced by DbDocGen
parser = HTMLSchemaParser("mydb_documentation.html")
schema_data = parser.parse()

# 2. Generate a diagram
generator = ERDGenerator(schema_data)

# PNG
generator.generate("output/mydb_erd.png", format="png")

# SVG
generator.generate("output/mydb_erd.svg", format="svg")

# PDF
generator.generate("output/mydb_erd.pdf", format="pdf")

# Draw.io with a color scheme
generator.generate("output/mydb_erd.drawio", format="drawio", color_scheme="Green")

# Lucidchart-compatible SVG
generator.generate("output/mydb_erd_lucid.svg", format="lucidchart")

# Visio-compatible SVG
generator.generate("output/mydb_erd_visio.svg", format="visio")
```

Full two-step workflow in Python

```
# Step 1 – generate HTML with DbDocGen
import sys
sys.path.insert(0, "../DbDocGen")
from db_parser import DatabaseParser
from db_doc_generator import DatabaseDocGenerator
from db_html_generator import HTMLGenerator

parser = DatabaseParser(db_type="postgresql", host="localhost", port=5432,
                        database="mydb", username="user", password="pass")
schema = parser.parse_schema(include_tables=True, include_views=True,
                             include_relationships=True)
doc = DatabaseDocGenerator(schema).generate_documentation(include_er_diagram=True)
with open("mydb_documentation.md", "w", encoding="utf-8") as f:
    f.write(doc)
html_path = HTMLGenerator("mydb_documentation.md").generate_html()

# Step 2 – generate ERD from that HTML
from erd_html_parser import HTMLSchemaParser
from erd_diagram_generator import ERDGenerator

schema_data = HTMLSchemaParser(html_path).parse()
ERDGenerator(schema_data).generate("mydb_erd.png", format="png")
```

Troubleshooting

“Graphviz not installed” / failed to execute 'dot'

The Graphviz system binary is missing or not on the PATH.

1. Verify whether it is installed: run `dot -V` in a terminal.
2. If not found, install it (see [Installation](#)).
3. If it is installed but not found, add the `bin` folder to your PATH:
 - Default Windows location: `C:\Program Files\Graphviz\bin`
 - Set it via **System** → **Advanced system settings** → **Environment Variables** → **Path**.
4. Open a new terminal after any PATH change.

[Draw.io](#), Lucidchart, and Visio formats do not need `dot` and will work without Graphviz.

“No tables found”

- Confirm the input HTML was generated by DbDocGen (not a hand-crafted file).
- Open the HTML in a browser and verify it contains a **Tables** section with column detail tables.
- If the file was generated with **Include Tables** unchecked in DbDocGen, re-generate it with that option enabled.

Diagram is generated but has no relationships / arrows

- Re-generate the DbDocGen HTML with **Include Relationships** checked.
- Confirm your database actually has foreign key constraints defined — without FK constraints, there are no relationships to draw.

The output file is very large or hard to read

- For large schemas (20+ tables), switch to **SVG** or **PDF** format — these are vector-based and scale cleanly.
- Use **DRAWIO** format to manually rearrange crowded tables after generation.

Import errors when running `erdgen.py`

```
pip install beautifulsoup4 graphviz lxml
```

Graphviz is installed but Python still can't find it (Windows)

Set the PATH temporarily in your terminal session before running:

```
set PATH=%PATH%;C:\Program Files\Graphviz\bin  
python erdgen.py
```

To make the change permanent, add the path via **System Properties** → **Environment Variables** and restart VS Code / your terminal.